

TUGAS AKHIR - EE234899

**NAVIGASI ROBOT BERBASIS LAYERED COSTMAP
UNTUK PERGERAKAN ANTAR LANTAI PADA
LINGKUNGAN INDOOR PASCA GEMPA BUMI**

BAGAS SURYA WIRAWAN

NRP 5022221026

Dosen Pembimbing

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Program Studi Strata 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



TUGAS AKHIR - EE234899

**NAVIGASI ROBOT BERBASIS LAYERED COSTMAP
UNTUK PERGERAKAN ANTAR LANTAI PADA
LINGKUNGAN INDOOR PASCA GEMPA BUMI**

BAGAS SURYA WIRAWAN

NRP 5022221026

Dosen Pembimbing

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Program Studi Strata 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]



FINAL PROJECT - EE234899

***ROBOT NAVIGATION USING LAYERED COSTMAP FOR
INTER-FLOOR MOVEMENT IN POST-EARTHQUAKE
INDOOR ENVIRONMENTS***

BAGAS SURYA WIRAWAN

NRP 5022221026

Advisor

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Undergraduate Study Program of Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : BAGAS SURYA WIRAWAN
Judul Tugas Akhir : NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI
Pembimbing : 1. Dr. Ir. Djoko Purwanto, M.Eng.
2. Fajar Budiman, S.T., M.Sc.

Indonesia, sebagai negara yang mengalami frekuensi bencana gempa bumi yang tinggi, seringkali harus melakukan misi Search and Rescue (SAR) di lingkungan indoor yang berbahaya, menimbulkan risiko signifikan bagi petugas. Untuk mengurangi risiko tersebut, robot otonom dapat berperan dalam menggantikan peran manusia pada kondisi tertentu. Untuk dapat melakukan misinya secara otonom, robot perlu memiliki kemampuan navigasi yang dapat bekerja pada lingkungan pasca gempa yang bertingkat. Proposal tugas akhir ini membahas pengembangan metode navigasi menggunakan *layered costmap* supaya robot dapat melintasi lingkungan tersebut. Skema ini merepresentasikan lingkungan bertingkat sebagai kumpulan *costmap* berlapis. Tiap layer pada *costmap* menyimpan informasi krusial seperti rintangan statis, area *obstacle dinamis*, zona inflasi, dan zona transisi antar *map*, yang memungkinkan navigasi komprehensif. Dalam arsitektur navigasi yang dikembangkan, setiap lantai atau tingkat pada lingkungan *indoor* direpresentasikan secara unik oleh satu *map* individual. Pendekatan ini secara efektif menyederhanakan informasi lingkungan 3D menjadi representasi 2D yang dapat dikelola, sehingga memungkinkan penggunaan sensor *Lidar* 2D untuk pengumpulan data lingkungan robot. Algoritma navigasi kemudian mengintegrasikan dan menggabungkan *map-map* 2D individual ini menjadi satu kesatuan yang kohesif, memungkinkan robot untuk memahami dan menavigasi lingkungan multi-lantai secara komprehensif. Sistem ini memungkinkan robot untuk berpindah dari satu *map* ke *map* lain, sesuai dengan posisi fisiknya di antara lantai-lantai yang berbeda dalam bangunan. Algoritma navigasi ini akan diuji coba di lingkungan laboratorium yang dirancang untuk mensimulasikan kondisi pasca gempa.

Kata Kunci: *Navigasi, Costmap, Robot, Antar lantai*

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : BAGAS SURYA WIRAWAN
Title : ROBOT NAVIGATION USING LAYERED COSTMAP FOR INTER-FLOOR MOVEMENT IN POST-EARTHQUAKE INDOOR ENVIRONMENTS
Advisors : 1. Dr. Ir. Djoko Purwanto, M.Eng.
2. Fajar Budiman, S.T., M.Sc.

Indonesia is a country experiencing high frequency of earthquake disasters, and so often needs to conduct Search and Rescue (SAR) missions in hazardous indoor environments, which pose significant risks to personnel. To mitigate these risks, autonomous robots can play a role in replacing human personnel in certain conditions. To carry out its mission autonomously, a robot needs navigation capabilities that can operate in multi-story post-earthquake environments. This final project proposal discusses the development of a navigation method using a layered costmap to enable robots to traverse such complex environments. This scheme represents multi-story environments as a collection of layered costmaps. Each layer in the costmap stores crucial information such as static obstacles, dynamic obstacles, inflation zones, and transition regions between maps, enabling comprehensive navigation. In the developed navigation architecture, each floor or level in the indoor environment is uniquely represented by an individual map. This approach effectively simplifies 3D environmental information into a manageable 2D representation, thus allowing the use of a 2D Lidar sensor for robot environmental data collection. The navigation algorithm then intelligently integrates and combines these individual 2D maps into a cohesive whole, enabling the robot to comprehensively understand and navigate multi-floor environments. This system allows the robot to seamlessly transition from one map to another, corresponding to its physical position between different floors within the building. This navigation algorithm will be tested in a laboratory environment designed to simulate post-earthquake conditions.

Keywords: Navigation, Costmap, Robot, Inter-floor.

[Halaman ini sengaja dikosongkan]

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena dengan rahmat dan karunia-Nya penulis dapat menyelesaikan penyusunan tugas akhir yang berjudul "NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI" dengan baik dan tepat waktu.

Surabaya, Juni 2026

BAGAS SURYA WIRAWAN

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

ABSTRAK	i
ABSTRACT	iii
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR GAMBAR	ix
DAFTAR TABEL	xi
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan	2
2 TINJAUAN PUSTAKA	5
2.1 Hasil Penelitian/Perancangan Terdahulu	5
2.1.1 A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2	5
2.1.2 A Slope-Adaptive Navigation Approach for Ground Mobile Robots . .	5
2.1.3 Path Planning of a Mobile Delivery Robot Operating in a Multi-Story Building Based on a Predefined Navigation Tree	5
2.1.4 ATV Navigation in Complex and Unstructured Environment Contain- ing Stairs	6
2.1.5 A Procedure for Taking a Remotely Controlled Elevator with an Au- tonomous Mobile Robot Based on 2D LiDAR	6
2.1.6 Development of an Indoor Delivery Mobile Robot for a Multi-Floor Environment	6
2.1.7 Autonomous Floor Navigation Control of Mobile Robot Using Elevator Button Recognition with Deep learning	7
2.1.8 Autonomous Traversing across Floors Based on Vision for reconfig- urable tracked robot	7

2.1.9	Mobile Service Robot Multi-floor Navigation Using Visual Detection and Recognition of Elevator Features	7
2.1.10	Autonomous Elevator Button Recognition and Operation Framework for Multi-Floor Mobile Manipulator Navigation	8
2.1.11	MuNES: Multifloor Navigation Including Elevators and Stairs	8
2.2	Teori/Konsep Dasar	8
2.2.1	Occupancy Grid	8
2.2.2	Layered Costmap	8
2.2.3	Lokalisasi Iterative Closest Point (ICP)	9
2.2.4	Path Planning A*	10
2.2.5	Path Tracking Pure Pursuit	10
3	METODOLOGI	13
3.1	Pengolahan Data Map	13
3.1.1	Map Region	13
3.1.2	Map Transisi	15
3.2	Perancangan Sistem Navigasi	15
3.2.1	Lokalisasi Iterative Closest Point (ICP)	15
3.2.2	Costmap	18
3.2.3	Local Planner	22
4	HASIL DAN PEMBAHASAN	25
4.1	Pengujian dan Hasil	25
4.1.1	Hasil Lokalisasi ICP	25
4.1.2	Hasil Global Planner	28
4.2	Pembahasan	33
5	PENUTUP	35
5.1	Kesimpulan	35
5.2	Saran	35
	DAFTAR PUSTAKA	37
	LAMPIRAN	39
	BIOGRAFI PENULIS	41

DAFTAR GAMBAR

3.1	Lingkungan arena dalam 3D	13
3.2	Map Occupancy Grid masing-masing region	14
3.3	Map Gabungan	14
3.4	Map transisi antar region	15
3.5	Diagram Navigasi	15
4.1	Simulasi di Region A	25
4.2	Error ICP di Region A	26
4.3	Simulasi di Region A dengan rintangan lantai	26
4.4	Error ICP di Region A dengan rintangan lantai	27
4.5	Simulasi di Region C	27
4.6	Error ICP di Region C	28
4.7	Relasi path terhadap Cost Scaling Factor di Region A	28
4.8	Semua path di region A. ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100	29
4.9	Relasi path terhadap Cost Scaling Factor di Region B	29
4.10	Semua path di region B. ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100	30
4.11	Relasi path terhadap Cost Scaling Factor di Region C	31
4.12	Semua path di region C. ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100	32

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

3.1	Nilai konstanta costmap	18
4.1	Karakteristik Path di Region A berdasarkan Cost Scaling Factor	29
4.2	Karakteristik Path di Region B berdasarkan Cost Scaling Factor	30
4.3	Karakteristik Path di Region C berdasarkan Cost Scaling Factor	31
4.4	Perbandingan path terhadap ground truth di region A	32
4.5	Perbandingan path terhadap ground truth di region B	32
4.6	Perbandingan path terhadap ground truth di region C	33

[Halaman ini sengaja dikosongkan]

BAB I

PENDAHULUAN

1.1 Latar Belakang

Indonesia adalah negara yang sering sekali mengalami bencana gempa bumi. Secara geografis, Indonesia terletak di antara 4 lempeng tektonik, yaitu Lempeng Pasifik, Lempeng Eurasia, Lempeng Indo-Australia, dan Lempeng Laut Filipina. Lokasi Indonesia di antara 4 lempeng ini menyebabkan frekuensi terjadinya gempa bumi cukup tinggi. Berdasarkan data dari BMKG, rata-rata Indonesia mengalami 18 gempa bumi tiap hari (Sabtaji, 2020). Pasca gempa bumi, operasi *search and rescue* (SAR) akan dilakukan di daerah yang terdampak gempa bumi untuk menolong korban dan menanggulangi kerusakan. Seringkali petugas SAR mengalami ancaman terhadap keselamatannya dalam melaksanakan operasi SAR.

Robot bisa ikut berperan dalam membantu petugas SAR melaksanakan tugasnya. Robot UAV mampu memberi gambaran medan pada daerah terjadinya bencana, sedangkan robot darat yang *track* atau *quadruped* dapat membantu membawa barang berat dan membantu dalam hal lainnya pada saat area bencana tidak dapat dilewati kendaraan besar. Robot juga dapat melakukan misi yang terlalu berbahaya untuk dilakukan oleh manusia. Sebagai contoh, sebuah robot darat pernah digunakan pasca gempa di Jepang untuk menginspeksi bangunan yang mengalami kerusakan. Mengirim petugas manusia dinilai terlalu berbahaya karena atap bangunan yang hampir roboh, sehingga digunakan robot yang dikendalikan dari jarak jauh (Lin et al., 2022).

Dalam implementasinya, robot darat SAR dapat dikendalikan manual oleh operator atau bisa otonom. Walaupun robot yang dikendalikan operator memiliki algoritma yang lebih sederhana, robot otonom juga memiliki peran pada operasi SAR, terutama pada daerah di mana infrastruktur komunikasinya rusak (Zhang et al., 2023). Untuk bisa melakukan misi otonom di dalam bangunan, robot darat perlu bisa melakukan lokalisasi, menemukan jalur navigasi yang aman, dan mencapai posisi tujuan, pada lingkungan tidak terstruktur, seperti pada lingkungan indoor dengan banyak puing yang disebabkan oleh gempa. Meskipun beberapa robot mampu bernavigasi pada lingkungan tidak terstruktur, navigasi robot darat pada area indoor masih menjadi permasalahan untuk robot otonom.

Di dalam proposal tugas akhir ini, diusulkan sebuah metode untuk mencoba menjawab permasalahan tersebut. Pada implementasi navigasi untuk robot darat otonom, umum digunakan *costmap* 2D untuk menyimpan informasi lingkungan. Metode ini terbatas saat robot perlu mencapai lokasi yang tidak terletak pada bidang yang sama. Untuk membenahi ini, sebuah metode untuk menyederhanakan informasi lingkungan 3D menjadi direpresentasikan oleh beberapa bidang *costmap* 2D atau *layered costmap*. Tiap *map* dapat mewakili sebuah lantai di dalam bangunan atau lingkungan indoor. Sebuah layer dapat juga menunjukkan tangga penghubung antar lantai. Lalu, robot darat otonom dapat bergerak pada salah satu *map* tertentu, kemudian berpindah ke *map* lain sesuai posisinya di bangunan.

1.2 Rumusan Masalah

Berdasarkan latar belakang pada bagian sebelumnya, terdapat beberapa inti masalah yang perlu dibahas. Permasalahan tersebut dijabarkan sebagai berikut:

1. Bagaimana cara untuk merepresentasikan informasi lingkungan yang terdiri dari beberapa lantai menjadi *layered costmap* untuk bisa digunakan pada algoritma navigasi antar lantai?
2. Bagaimana cara membuat algoritma lokalisasi, *path planning*, dan *path tracking* yang bisa bekerja pada algoritma navigasi yang menggunakan *layered costmap*?
3. Apakah dengan menggunakan metode navigasi antarlantai, robot darat otonom dapat melakukan navigasi hingga mencapai tujuan pada arena lab?

1.3 Batasan Masalah

Dalam tugas akhir ini, metode multilayer *costmap* yang dikembangkan tidak dirancang untuk langsung digunakan di kondisi pasca gempa. Hal ini disebabkan oleh beberapa faktor berikut:

1. Pengujian dilakukan pada arena laboratorium yang didesain untuk merekayasa kondisi pasca gempa. Arena dilengkapi dengan rekayasa medan reruntuhan, tanjakan, dan permukaan tidak rata, yang dibuat pada skala lebih kecil.
2. Navigasi dilakukan oleh robot darat hexapod yang melakukan navigasi secara otonom, dengan bergerak dari titik awal menuju tujuan yang telah ditentukan.
3. Algoritma navigasi yang digunakan digunakan pada kondisi arena laboratorium yang sudah dimapping sebelumnya sehingga lokasi rintangan statis sudah diketahui.

1.4 Tujuan

Tugas akhir ini bertujuan untuk:

1. Mengembangkan metode representasi lingkungan dalam bentuk *layered costmap* untuk menyederhanakan informasi lingkungan yang terdiri dari beberapa lantai, yang dapat digunakan oleh robot otonom untuk navigasi antar lantai.
2. Merancang dan mengimplementasikan algoritma lokalisasi, *path planning*, dan *path tracking*, yang menggunakan *layered costmap* agar robot dapat bernavigasi antar lantai.
3. Melakukan pengujian sistem navigasi berbasis *layered costmap* di lingkungan laboratorium, untuk mengevaluasi efektivitas dan keterbatasan dari pendekatan yang diusulkan sebelum diterapkan pada skenario nyata.

Tugas akhir ini diharapkan dapat memberikan manfaat sebagai berikut:

1. Penelitian ini memberikan kontribusi terhadap pengembangan sistem navigasi robot otonom, khususnya dalam pemrosesan data lingkungan kompleks menjadi representasi yang lebih sederhana dan terstruktur.

2. Metode multilayer *costmap* yang diusulkan dapat menjadi dasar awal untuk navigasi robot di area bertingkat atau tidak rata, seperti bangunan runtuh pasca gempa.
3. Dasar untuk penelitian lanjutan dalam pengembangan sistem robot yang mampu beroperasi secara otonom di area terdampak bencana, dengan mempertimbangkan faktor medan nyata yang kompleks dan tidak terstruktur

[Halaman ini sengaja dikosongkan]

BAB II

TINJAUAN PUSTAKA

2.1 Hasil Penelitian/Perancangan Terdahulu

2.1.1 A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2

Penelitian oleh (Macenski et al., 2023) membahas metode *layered costmap*, yaitu sebuah metode untuk menambah lapisan secara sistematis dengan tiap lapisan mampu menyimpan informasi lingkungan tertentu. Tiap lapisan dapat mewakili sumber informasi, algoritma, ataupun hasil berbeda. Setiap siklus update dapat memperbarui informasi pada grid *costmap*. Manfaatnya dengan menggunakan *layered costmap* termasuk pembaruan yang lebih jelas, area pembaruan yang dinamis, proses pembaruan yang teratur, dan konfigurasi yang fleksibel, memungkinkan representasi nilai biaya yang kompleks dan perilaku navigasi yang peka konteks, seperti navigasi sosial. Implementasi ini telah terintegrasi dengan ROS 2 dan diadopsi sebagai algoritma navigasi default.

2.1.2 A Slope-Adaptive Navigation Approach for Ground Mobile Robots

Studi oleh (Hu et al., 2022) mengusulkan pendekatan navigasi adaptif kemiringan untuk robot bergerak darat, yang bertujuan mengatasi keterbatasan peta biaya 2-dimensi yang sering salah mengartikan kemiringan (slope) sebagai area terlarang. Pendekatan baru ini didasarkan pada peta biaya multi-lapis yang secara aktif mengintegrasikan informasi kemiringan selama tahap pembuatan peta lingkungan. Sebuah algoritma deteksi kemiringan dikembangkan untuk mengidentifikasi kemiringan dan secara adaptif mengubah peta biaya selama navigasi, sehingga kemiringan dianggap sebagai jalur yang dapat dilalui, hanya dengan biaya tambahan. Selain itu, untuk skenario di mana kemiringan mengarah ke lantai baru, sistem mengadopsi kode Aruco untuk memicu peralihan informasi peta dan sistem koordinat referensi yang sesuai, memungkinkan navigasi berlanjut tanpa gangguan. Kontribusi utama meliputi deteksi kemiringan, lapisan peta biaya kemiringan, dan navigasi 3-dimensi. Baik simulasi maupun hasil eksperimen di dunia nyata menunjukkan efektivitas tinggi dari pendekatan yang diusulkan ini, memastikan robot dapat menyelesaikan tugas navigasi di medan dengan karakteristik kemiringan secara lancar tanpa memengaruhi akurasi posisi.

2.1.3 Path Planning of a Mobile Delivery Robot Operating in a Multi-Story Building Based on a Predefined Navigation Tree

Penelitian oleh (Palacín, Rubies, et al., 2023) membahas metode perencanaan jalur untuk robot pengirim paket yang bisa bekerja pada bangunan bertingkat, sebuah masalah yang membutuhkan informasi spasial dan operasional. Paper ini menggunakan *static navigation tree* yang menggambarkan *weighted paths* dalam bangunan bertingkat. Lalu, *navigation tree* digunakan untuk merencanakan jalur dan mengestimasi durasi dari semua rute pengiriman menggunakan algoritma dijkstra. Metode ini dapat digunakan dengan asumsi berikut: (1) peta lantai bangunan sudah diketahui; (2) posisi titik awal dan tujuan sudah diketahui; (3) robot memiliki sensor

untuk lokalisasi diri; (4) Bangunan memiliki lift yang dapat dioperasi dengan remot; (5) semua pintu dalam keadaan terbuka. Hasil eksperimen menunjukkan bahwa metode ini dapat efektif merencanakan jalur optimal untuk robot pengirim paket di bangunan bertingkat.

2.1.4 ATV Navigation in Complex and Unstructured Environment Containing Stairs

Penelitian oleh (Zhu et al., 2020) mengusulkan metode *real-time* untuk mendeteksi dan lokalisasi tangga. Metode ini diterapkan pada robot *All-Terrain Vehicle* (ATV) yang sering digunakan untuk misi di daerah dengan lingkungan kompleks. Sensor LiDAR VLP-16 digunakan untuk mendeteksi fitur geometris dari tangga berupa bidang miring dan garis paralel. Sensor IMU 3-axis juga digunakan saat robot memanjat tangga dengan basis antara coupling antara roll dan yaw pada bidang miring. Eksperimen yang telah dilakukan menunjukkan bahwa metode ini dapat mendeteksi tangga dengan akurasi tinggi dan memungkinkan ATV untuk menavigasi tangga secara otonom.

2.1.5 A Procedure for Taking a Remotely Controlled Elevator with an Autonomous Mobile Robot Based on 2D LiDAR

Studi oleh (Palacín, Bitriá, et al., 2023) menyajikan prosedur bagi robot mobile otonom untuk bernavigasi antar lantai di gedung bertingkat dengan menggunakan lift yang dikontrol dari jarak jauh dengan menggunakan informasi dari sensor 2D LiDAR. Pendekatan ini menggunakan robot APR-02 di dalam eksperimen, untuk melakukan self-localization dengan mencocokkan scan 2D LiDAR saat ini dengan peta 2D yang sudah ada, proses yang didasarkan pada algoritma Iterative Closest Point (ICP). Peta tersebut mencakup *waypoint* yang secara spesifik mendefinisikan posisi setiap lift, baik di luar maupun di dalam kabin. Interaksi dengan lift, termasuk memanggil dan memilih lantai tujuan, dicapai melalui perintah jarak jauh yang dikirim ke perangkat *Internet of Things* (IoT) tambahan yang dikembangkan untuk mengontrol tombol lift asli tanpa manipulasi invasif. Prosedur ini melibatkan urutan aksi untuk masuk dan keluar lift, yang memerlukan robot untuk menavigasi jalur yang tepat dan mendeteksi status pintu lift dengan memantau titik scan 2D LiDAR dalam area persegi panjang yang ditentukan. Hasil eksperimental menunjukkan bahwa self-localization robot tidak terpengaruh signifikan oleh status pintu dan pelacakan jalur masuk dan keluar lift berhasil, meskipun keterbatasan teridentifikasi di mana kurangnya umpan balik lantai dari perangkat IoT dapat menyebabkan kesalahan jika lift berhenti mendadak. kemampuan robot untuk "membedakan lantai" bergantung pada gabungan pengetahuannya tentang peta multi-lantai, pemberian perintah lantai tujuan tertentu melalui sistem kontrol jarak jauh, dan asumsi bahwa robot berada di lantai yang benar ketika pintu lift terbuka.

2.1.6 Development of an Indoor Delivery Mobile Robot for a Multi-Floor Environment

Studi oleh (T. Kim et al., 2024) menguraikan pengembangan *indoor delivery mobile robot* dengan arsitektur yang dirancang khusus untuk lingkungan multi-lantai. Arsitektur yang diusulkan terdiri dari lima modul utama: manajemen peta, lokalisasi, perencanaan jalur, persepsi, dan perencanaan tugas. Untuk mengatasi tantangan navigasi di gedung bertingkat, robot ini menggunakan peta navigasi terintegrasi yang dibuat dengan menggabungkan peta *point cloud* multi-lantai dan peta topologi berbasis node graph, memungkinkan lokalisasi yang efektif dan perencanaan rute 3D yang mencakup pergerakan antar-lantai. Modul persepsi memungkinkan robot

untuk mendeteksi dan naik turun elevator secara mandiri melalui ekstraksi area yang dapat dilalui, sementara modul perencanaan tugas mengelola berbagai perilaku yang diperlukan untuk layanan pengiriman, seperti mengemudi, naik/turun elevator, pengiriman, dan docking. Efektivitas dan ketahanan arsitektur ini telah berhasil ditunjukkan melalui uji lapangan selama sebulan di gedung biasa selama jam kerja.

2.1.7 Autonomous Floor Navigation Control of Mobile Robot Using Elevator Button Recognition with Deep learning

Penelitian oleh (Lee et al., 2023) memperkenalkan sistem kontrol navigasi lantai otonom untuk robot bergerak yang memanfaatkan pengenalan tombol lift dengan *deep learning* untuk memungkinkan robot beroperasi secara mandiri di antara lantai tanpa memerlukan modifikasi fasilitas lift yang ada atau sistem manajemen gedung tambahan. Sistem yang diusulkan menggunakan deep neural network yang dilatih dengan dataset besar gambar tombol lift untuk secara akurat mengenali dan menentukan posisi tombol. Sebuah robot bergerak, dilengkapi dengan kamera OAK-D dan manipulator myCobot 280, mampu menavigasi ke lift, mendeteksi dan menekan tombol yang diinginkan (baik tombol naik/turun maupun tombol lantai tujuan), memasuki lift, berputar di dalamnya, dan keluar di lantai yang benar. Efektivitas dan keandalan sistem ini telah berhasil dibuktikan melalui uji coba di lingkungan nyata, dengan tingkat keberhasilan tinggi (7 dari 8 percobaan berhasil) dalam memindahkan robot dari lantai empat ke lantai dua.

2.1.8 Autonomous Traversing across Floors Based on Vision for reconfigurable tracked robot

Studi oleh (Xie & Zhou, 2023) menyajikan sistem navigasi otonom *full-stack* untuk *reconfigurable tracked robot* yang dirancang khusus untuk melintasi lantai dan tangga dengan aman di lingkungan buatan manusia. Sistem ini mengintegrasikan algoritma A* untuk perencanaan jalur awal dan *DWA-based path following* untuk permukaan datar. Untuk navigasi tangga, robot menggunakan algoritma deteksi tangga berbasis visi dari data gambar kedalaman kamera RGB-D untuk mengekstrak parameter tangga seperti tinggi dan lebar langkah. Kemudian, strategi gerakan yang terbagi dalam empat tahap diterapkan untuk melintasi tangga secara mandiri, termasuk penyesuaian sudut flipper dan penggunaan kontroler PD untuk mempertahankan orientasi tegak lurus terhadap tangga dan mencegah selip. Robot ini dilengkapi dengan *Flipper Auxiliary Rotation System (FARS)* yang meningkatkan kemampuan melintasi rintangan melalui penyesuaian bentuk trek. Validasi eksperimental di lingkungan nyata (tangga indoor dan outdoor) menunjukkan bahwa pendekatan ini memungkinkan robot menyelesaikan tugas lintasan antar-lantai secara stabil, mulus, dan aman, bahkan mampu mengoreksi deviasi arah akibat selip.

2.1.9 Mobile Service Robot Multi-floor Navigation Using Visual Detection and Recognition of Elevator Features

Paper oleh (E.-H. Kim et al., 2020) membahas navigasi multi-lantai robot layanan seluler, khususnya dalam hal masuk dan keluar lift. Penulis mengusulkan metode deteksi dan pengenalan fitur lift serta navigasi robot menggunakan pembelajaran mendalam. Pendekatan ini melibatkan dua metode: ekstraksi koordinat tombol lift melalui ekstraktor fitur tradisional seperti adaptive thresholding, blob detection, dan template matching, serta pengenalan berbasis *deep learning* menggunakan YOLO untuk panel display penghitung lantai. Hasil analisis

menunjukkan bahwa ekstraktor fitur tradisional mengungguli sistem pengenalan berbasis *deep learning* dalam kondisi sulit seperti pantulan cahaya dan motion blur, menjadikannya sistem yang lebih kuat untuk deteksi dan pengenalan. Perangkat keras robot meliputi dua kamera, lengan robot, LiDAR, dan router, dengan setiap kamera memiliki fungsi spesifik dalam deteksi tombol dan pengenalan panel display. Sistem kontrol robot secara keseluruhan terdiri dari modul Persepsi, Koordinasi, dan Eksekusi. Eksperimen navigasi robot berhasil memverifikasi metode yang diusulkan dalam lingkungan nyata.

2.1.10 Autonomous Elevator Button Recognition and Operation Framework for Multi-Floor Mobile Manipulator Navigation

Paper oleh (Li et al., 2021) mengusulkan Kerangka kerja Autonomous Elevator Button Recognition and Operation (AEBRO) yang memungkinkan manipulator bergerak beroda menavigasi gedung bertingkat dengan mengoperasikan lift secara otonom. Kerangka kerja ini terdiri dari tiga fase berurutan: Fase I berfokus pada deteksi Region of Interest (ROI), menggunakan metode berbasis wilayah konektif dengan detektor Maximally Stable Extremal Regions (MSER) untuk menemukan teks angka dan algoritma klasterisasi untuk mengusulkan posisi tombol. Fase II melibatkan pengenalan teks angka ini menggunakan model jaringan saraf konvolusional (CNN) berskala kecil. Terakhir, Fase III menangani pengoperasian tombol lift, mengintegrasikan mekanisme koreksi kesalahan berdasarkan tata letak spasial dan kontinuitas angka untuk memperbaiki deteksi positif palsu dan memprediksi tombol yang tidak terdeteksi, diikuti oleh kontrol manipulator yang dipandu oleh data kamera penginderaan kedalaman. Evaluasi pada kumpulan data panel tombol lift yang bervariasi dan eksperimen dunia nyata menunjukkan bahwa kerangka kerja AEBRO mencapai F1-score tinggi sebesar 0,985, mengungguli metode mutakhir dalam akurasi dan ketahanan di berbagai lift dan kondisi, dengan kecepatan pengenalan (dalam 0,25 detik) yang sebanding dengan persepsi manusia.

2.1.11 MuNES: Multifloor Navigation Including Elevators and Stairs

Paper oleh (Jung et al., 2024) mengusulkan skema untuk pemetaan tunggal *multifloor* dan perencanaan lintasan bagi robot mobil, termasuk penggunaan lift dan tangga, untuk mengatasi ketiadaan peta *multifloor* yang cocok dalam metode yang ada. Sistem ini menciptakan peta *multifloor* tunggal dengan mengestimasi perubahan ketinggian berdasarkan data tekanan dan melakukan deteksi loop berbasis rantai untuk akurasi yang lebih baik. Peta kemudian di-"voxelize" untuk perencanaan lintasan optimal dan realistis. Algoritma A* digunakan dengan fungsi biaya yang mempertimbangkan faktor-faktor realistis seperti waktu tunggu lift, memungkinkan robot memilih mode pergerakan antar lantai (lift atau tangga) secara efisien berdasarkan titik awal, titik akhir, dan waktu tunggu lift. Pendekatan ini mengatasi keterbatasan metode sebelumnya yang kesulitan memetakan di dalam lift atau memiliki akumulasi error odometri dari sensor lain seperti IMU.

2.2 Teori/Konsep Dasar

2.2.1 Occupancy Grid

2.2.2 Layered Costmap

Costmap adalah representasi grid dua dimensi dari lingkungan robot yang digunakan dalam sistem navigasi, dengan tiap sel pada grid memiliki harga yang terasosiasi dengannya. Pada *costmap* modern, tiap sel merupakan integer dari 0 hingga 255 yang merepresentasikan biaya

navigasi. Secara umum, algoritma perencanaan jalur akan mengutamakan melewati sel dengan harga rendah dan menghindari sel dengan harga tinggi. Metode *costmap* memberikan keseimbangan antara kualitas informasi dan efisiensi algoritma. Sebagai ekstensi dari *costmap*, digunakan konsep *layered costmap*, yaitu struktur costmap yang terbagi menjadi beberapa layer terpisah berdasarkan jenis data atau fungsi spesifik (misalnya static map, obstacle, inflation, proxemic). Setiap layer memodifikasi master costmap secara terstruktur dan independen, memungkinkan sistem navigasi mempertimbangkan konteks yang lebih kompleks dan menghasilkan perilaku pergerakan robot yang lebih cerdas dan adaptif terhadap lingkungan (Macenski et al., 2023).

2.2.3 Lokalisasi Iterative Closest Point (ICP)

Lokalisasi robot adalah proses untuk menentukan posisi dan orientasi robot (pose) relatif terhadap peta lingkungan tempat robot beroperasi. Kemampuan ini sangat penting bagi robot untuk melakukan navigasi dan pengambilan keputusan yang tepat dalam menjalankan misi otonom. Lokalisasi umumnya menggunakan informasi dari sensor internal, seperti encoder roda dan IMU, dan sensor eksternal, seperti kamera atau LiDAR, untuk memperkirakan posisi robot berdasarkan model gerak dan observasi lingkungan.

Salah satu kategori metode lokalisasi adalah map-matching, dimana data sensor dari lingkungan dicoba untuk disamakan dengan fixed map yang sudah ada. Algoritma iterative closest point (ICP) merupakan salah satu algoritma yang bekerja dengan metode map-matching ini. Cara kerja algoritma ICP adalah dengan mencoba mengurangi jarak Euclidean antara input data dengan sebuah model referensi (dalam lokalisasi adalah fixed map). Secara matematis, bayangkan terdapat dua set titik 2D A dan B. Tujuan algoritma adalah menemukan sebuah transformasi A ke B untuk mencari mean squared distance (MSD) antara titik A dan B (Sobreira et al., 2019).

$$MSD(A, B, u) = \frac{1}{n} \sum_{a \in A, b \in B} \|b - u(a)\|^2 \quad (2.1)$$

Dengan rumus matematika diatas, algoritma ICP mencoba untuk mengurangi MSD pada setiap iterasi. ICP terbagi menjadi dua tahap, yaitu matching stage dan transformation stage. Matching stage adalah tahap pertama dari ICP dan bertujuan untuk meminimalkan nilai mean square distance dengan dengan matching terbaik antara set titik A dan B.

Transformation stage adalah tahap kedua yang bertujuan untuk mencari nilai paling optimal dari transformasi rotasi R dan translasi t untuk mencapai MSD paling kecil. ICP menggunakan simple least square solver untuk menemukan transformasi matrix (R — t) yang meminimalkan MSD. untuk mencapai hal tersebut dilakukan pengurangan dari data referensi dan point cloud sensor nilai centroid masing-masing, yang ditunjukkan di rumus dibawah (Sobreira et al., 2019).

$$a'_i = a_i - \frac{1}{n} \sum_{i=0}^n a_i \quad (2.2)$$

$$b'_i = b_i - \frac{1}{m} \sum_{i=0}^m b_i \quad (2.3)$$

Setelah itu, nilai kovarian silang matrix dikomputasi menggunakan persamaan dibawah

dengan A' dan B' sama dengan set poin a'_i dan b'_i .

$$H = A' B'^T \quad (2.4)$$

Lalu, sudut rotasi θ bisa dihitung dengan rumus dibawah .

$$\theta = \text{atan2}((H(0, 1) - H(1, 0)), (H(0, 0) + H(1, 1))) \quad (2.5)$$

Pada akhir tahap transformasi, data sensor diubah menggunakan matriks (R — t) yang diperkirakan dan algoritma kembali ke tahap pencocokan, kecuali jika kriteria penghentian diverifikasi (misalnya, jumlah iterasi, peningkatan kesalahan Euclidean antar iterasi, dll.) (Sobreira et al., 2019).

$$R = m \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (2.6)$$

$$t = \bar{b} - R\bar{a} \quad (2.7)$$

2.2.3.1 KNN

2.2.4 Path Planning A*

Path planning adalah proses perencanaan jalur geometris yang menghubungkan titik awal ke titik tujuan tanpa menabrak rintangan, biasanya dilakukan dalam ruang konfigurasi (configuration space) dengan mempertimbangkan berbagai batasan lingkungan. Metode *path planning* umum mencakup teknik roadmap, cell decomposition, dan artificial potential field.

Salah satu algoritma *path planning* yang paling sering digunakan adalah A*. A* dikategorikan sebagai algoritma best first search (BFS) yang menggabungkan kelebihan uniform-cost dan greedy searches dengan menggunakan fitness function.

$$f(n) = g(n) + h(n) \quad (2.8)$$

Pada persamaan diatas, $g(n)$ adalah biaya kumulatif dari titik awal ke simpul n , dan $h(n)$ adalah estimasi heuristik biaya tersisa menuju simpul tujuan. Selama pencarian, A* mengelola daftar terbuka (simpul yang akan dipertimbangkan) dan daftar tertutup (simpul yang sudah dikunjungi). Algoritma ini bekerja dengan memperluas simpul dari daftar terbuka dengan fungsi kecocokan minimal, memindahkannya ke daftar tertutup, dan menambahkan tetangganya ke daftar terbuka hingga simpul tujuan diperluas.

Pilihan heuristik yang baik sangat penting untuk kualitas dan efisiensi pencarian A*. Heuristik yang mengestimasi biaya nyata menjamin jalur terpendek, tetapi bisa memperluas terlalu banyak simpul. Sebaliknya, heuristik yang melebih-lebihkan dapat mempercepat pencarian ke tujuan, namun berisiko melambatkan jika jalur optimal menyimpang. Meskipun A* banyak digunakan karena menemukan jalur biaya minimum dan memastikan keberadaan jalur bebas, kelemahan utamanya adalah konsumsi waktu yang tinggi dan berat komputasi yang signifikan (Damle & Susan, 2022).

2.2.5 Path Tracking Pure Pursuit

Path tracking atau trajectory planning bertugas menambahkan informasi waktu pada jalur tersebut, sehingga menghasilkan lintasan yang dapat diikuti oleh robot secara fisik. Proses ini

mempertimbangkan batasan kinematik dan dinamik dari robot, termasuk kecepatan, percepatan, dan jerk, untuk menghasilkan lintasan yang halus, aman, dan efisien. *path tracking* memastikan bahwa gerakan aktual robot mengikuti jalur yang telah direncanakan dengan akurat dan memastikan kestabilan robot serta performa yang sesuai dengan sistem kontrol yang digunakan (Yao et al., 2020).

Pure-pursuit adalah metode *path tracking* geometris yang efektif untuk robot otonom. Cara kerjanya adalah membayangkan kendaraan mengikuti sebuah titik di jalur yang diberikan, beberapa jarak di depannya. Titik ini dipilih pada jarak pandang ke depan (look-ahead distance) yang ditentukan. Implementasinya meliputi penentuan lokasi kendaraan saat ini, menemukan titik terdekat di jalur, memilih titik tujuan pada jarak pandang ke depan konstan, mengubah koordinat titik tujuan ke kendaraan, menghitung kelengkungan, dan memperoleh sudut kemudi. Keunggulan metode ini adalah kemudahan penyetelan jarak pandang ke depan, kesederhanaan komputasi, dan ketiadaan istilah turunan (Yao et al., 2020).

$$\text{Error}_{\text{cte}} = \text{Error}_{\text{calculate}} + \text{Error}_{\text{tracking}} \quad (2.9)$$

[Halaman ini sengaja dikosongkan]

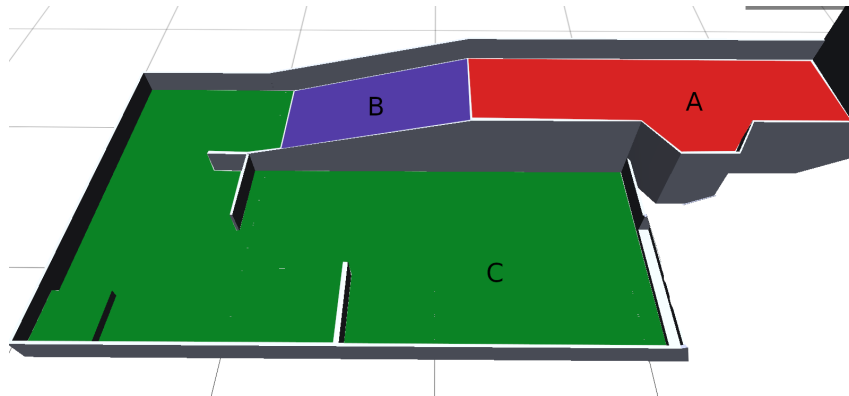
BAB III

METODOLOGI

3.1 Pengolahan Data Map

Sistem navigasi ini dirancang untuk beroperasi pada area yang petanya sudah diketahui sebelumnya (*known map*). Informasi lingkungan tersebut disimpan dalam bentuk *occupancy grid*. Tantangan utama yang dihadapi adalah bahwa data lingkungan yang dikumpulkan bersifat tiga dimensi (3D), sementara *occupancy grid* yang digunakan hanya memiliki dua dimensi (2D). Untuk menyetarakan perbedaan ini, lingkungan 3D dibagi menjadi beberapa *region* yang lebih kecil, dan setiap *region* direpresentasikan secara terpisah dalam bentuk peta 2D. Sehingga, setiap *region* menjadi proyeksi 2D dari sub-bagian lingkungan 3D. Pembagian *region* dalam ruang 3D dapat dilihat pada Gambar 3.1.

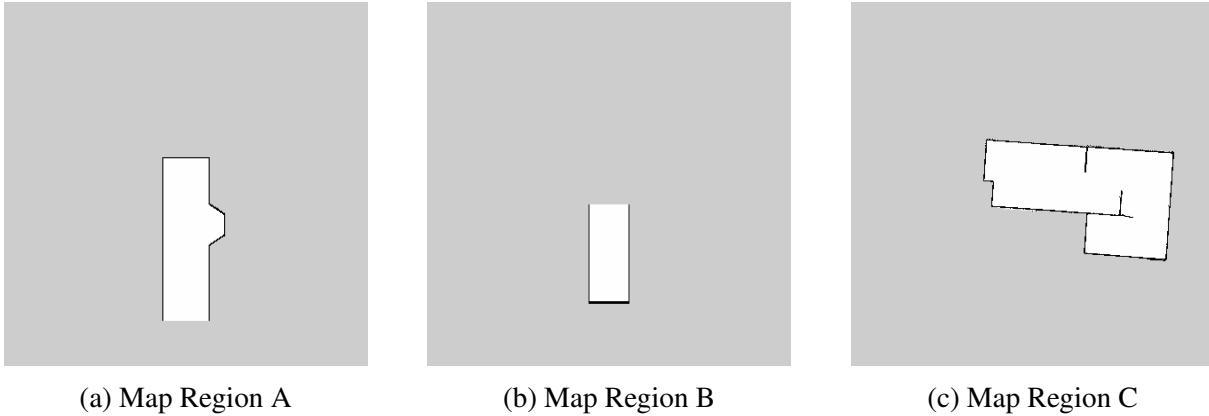
Selain peta utama, sistem navigasi juga membutuhkan peta transisi yang berfungsi sebagai jembatan penghubung antar *region*. Peta transisi adalah peta terpisah yang menyimpan informasi relasi spasial setiap *region* terhadap *global frame of reference* yang seragam, sehingga posisi absolut masing-masing *region* dalam koordinat global dapat diketahui. Dengan adanya peta transisi ini, robot dapat melakukan perpindahan antar *region* secara mulus selama navigasi, meskipun kedua *region* terpisah oleh area *unknown*.



Gambar 3.1: Lingkungan arena dalam 3D

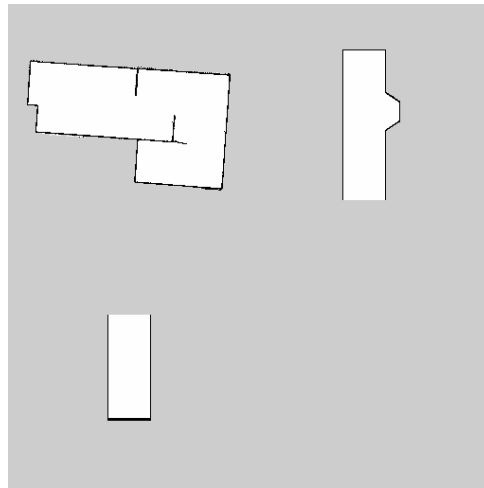
3.1.1 Map Region

Peta untuk setiap *region* diperoleh melalui proses *mapping* menggunakan Hector SLAM atau dibuat secara manual berdasarkan dimensi ukuran lingkungan asli. Hasil akhir peta masing-masing *region* ditunjukkan pada Gambar 3.2.



Gambar 3.2: Map Occupancy Grid masing-masing region

Sistem navigasi tidak menggunakan peta individu dari masing-masing *region* secara terpisah, melainkan menggunakan satu peta utama yang merupakan hasil kombinasi seluruh *region*. Dalam peta utama tersebut, setiap *region* dipisahkan oleh area *unknown* (wilayah yang tidak diketahui).



Gambar 3.3: Map Gabungan

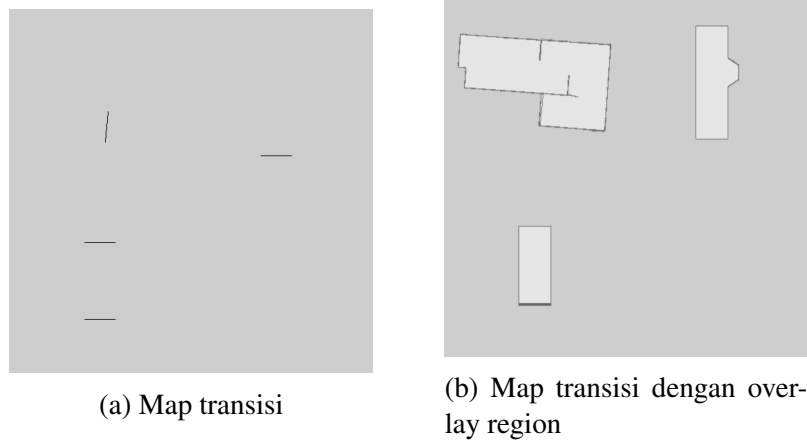
Untuk menghasilkan map gabungan, diperlukan deteksi Region of Interest (ROI) pada setiap peta pisah. ROI didefinisikan sebagai area yang mengandung piksel gelap (nilai $\geq$ threshold), yang mewakili obstacle atau peta yang bermakna. Untuk setiap peta, ROI diidentifikasi melalui:

$$\text{ROI} = \{(x, y) : I(x, y) < T\}$$

dimana, $I(x, y)$ adalah nilai intensitas piksel pada koordinat (x, y) , T adalah threshold

Peta gabungan akhir tersusun dari beberapa bagian, dan setiap bagian ditempati oleh satu *region*. Banyaknya bagian tersebut ditentukan oleh jumlah peta *region* yang dimasukkan ke dalam proses penggabungan.

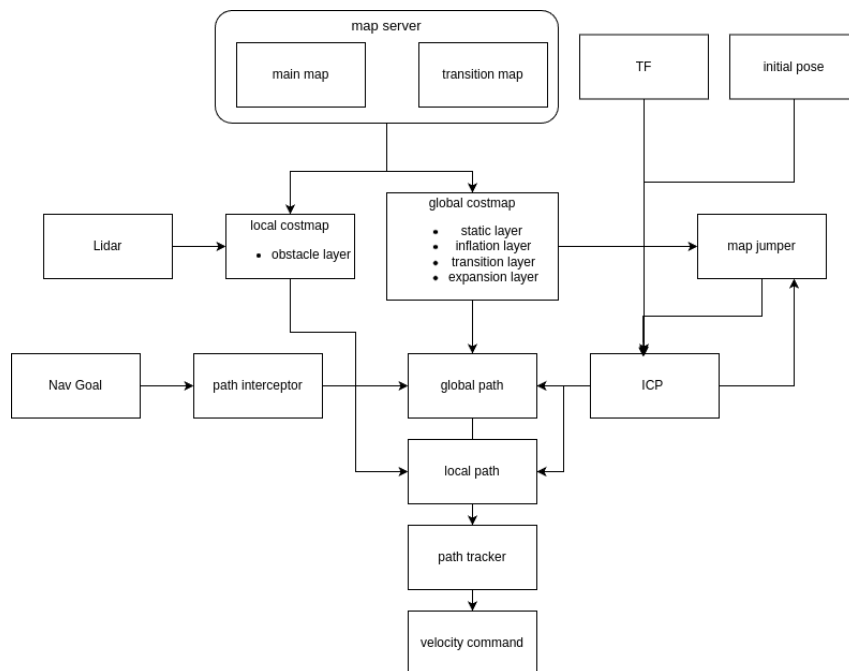
3.1.2 Map Transisi



Gambar 3.4: Map transisi antar region

Peta transisi digunakan untuk menandakan perbedaan antara

3.2 Perancangan Sistem Navigasi



Gambar 3.5: Diagram Navigasi

3.2.1 Lokalisasi Iterative Closest Point (ICP)

3.2.1.1 Prekomputasi KNN

Sistem ini melakukan pre-komputasi atas semua korespondensi saat menerima *map* baru lalu menyimpan informasi tersebut di dalam tabel hash. Ini memungkinkan karena *occupancy*

grid sudah diketahui dan bersifat diskrit, sehingga dapat diketahui relasi antara sel bebas yang berada di dekat tembok dengan K *nearest* sel rintangan.

Untuk setiap sel rintangan pada indeks i_{occ} :

1. Cek kejangkauan menggunakan *Moore neighbourhood* 3x3 untuk mengecek keberadaan sel bebas di sekitarnya. Jika tidak ada sel bebas, maka sel rintangan tersebut tidak dapat dijangkau dan diabaikan.
2. Disekitar indeks i_{occ} yang memenuhi syarat sebelumnya, dilakukan iterasi mencari semua sel dengan radius r
3. Untuk setiap cell tersebut pada index i_{nb} : Insert posisi dari i_{occ} ke dalam daftar KNN milik i_{nb} . Jaga urutan terurut berdasarkan Euclidean distance

3.2.1.2 Proses Scan

Untuk scan dengan N beams pada sudut $\theta_0, \theta_1, \dots, \theta_{N-1}$:

$$\mathbf{C} = \begin{bmatrix} \cos \theta_0 & \cos \theta_1 & \cdots & \cos \theta_{N-1} \\ \sin \theta_0 & \sin \theta_1 & \cdots & \sin \theta_{N-1} \end{bmatrix}_{2 \times N}$$

Cache hanya dihitung ulang ketika parameter scan (jumlah beams, rentang sudut, increment) berubah.

Diberikan range readings $r_0, r_1, \dots, r_{N-1}$:

$$\mathbf{S} = \begin{bmatrix} r_0 \cos \theta_0 & r_1 \cos \theta_1 & \cdots \\ r_0 \sin \theta_0 & r_1 \sin \theta_1 & \cdots \end{bmatrix}$$

3.2.1.3 Loop Utama

Algorithm 1 ICP Single Iteration

Require: <i>scan</i> Ensure: <i>new_to_old</i> 1: $T \leftarrow \text{Identity}$ 2: for <i>iteration</i> = 1 to <i>max_iter</i> do 3: $\text{random_scan} \leftarrow \text{sampler}(\text{scan})$ 4: $\text{data} \leftarrow T \cdot \text{random_scan}$ 5: $M \leftarrow \text{matches}(\text{data}, \text{knn})$ 6: $w \leftarrow \text{get_weights}(M)$ 7: $T_update \leftarrow \text{point_to_map}(M, w)$ 8: $T \leftarrow T_update \cdot T$ 9: if $\text{converged}(T_update)$ then 10: break 11: end if 12: end for 13: return T	▶ N points dalam sensor frame ▶ transform_t ▶ subsample dengan random stride ▶ transform ke map frame ▶ cari correspondences ▶ weight setiap match ▶ SVD solver ▶ akumulasi
---	--

3.2.1.4 Correspondence Matching

Untuk setiap sensor point s_j :

1. Cek apakah s_j berada di dalam map bounds: `converter.valid_position(s_j)`
2. Konversi posisi ke flat index: $i = \text{converter.to_index}(s_j)$
3. Lookup `knn_[i]` — sebuah sorted vector berisi hingga K occupied map points
4. Duplikasi s_j untuk setiap nearest neighbors-nya, menghasilkan $(s_j, m_{j,1}), (s_j, m_{j,2}), \dots, (s_j, m_{j,K})$

Ini memperluas N sensor points menjadi hingga $N \times K$ pasangan yang cocok.

3.2.1.5 Solusi SVD

Diberikan M pasangan $\{s_i\}$ (sensor) dan $\{m_i\}$ (map) dengan weights $\{w_i\}$, kita mencari rigid transform $T \in SE(2)$ yang meminimalkan:

$$T^* = \arg \min_{R, t} \sum_{i=1}^M w_i \|m_i - (Rs_i + t)\|^2$$

Langkah 1: Weighted Centroids

$$\bar{s} = \frac{\sum_{i=1}^M w_i s_i}{\sum_{i=1}^M w_i}, \quad \bar{m} = \frac{\sum_{i=1}^M w_i m_i}{\sum_{i=1}^M w_i}$$

Langkah 2: Centering (pindahkan titik ke pusat koordinat)

$$s'_i = s_i - \bar{s}, \quad m'_i = m_i - \bar{m}$$

Langkah 3: Cross-Covariance Matrix

$$H = \sum_{i=1}^M w_i m'_i (s'_i)^\top = M W S^\top$$

dimana $S, M \in \mathbb{R}^{2 \times M}$ adalah matriks yang kolomnya berisi titik-titik yang sudah di-center, dan $W = \text{diag}(w_1, \dots, w_M)$.

Langkah 4: Singular Value Decomposition

$$H = U \Sigma V^\top$$

dengan $U, V \in \mathbb{R}^{2 \times 2}$ orthonormal dan $\Sigma = \text{diag}(\sigma_1, \sigma_2)$.

Langkah 5: Rotasi Optimal

$$R = UV^\top$$

Jika $\det(R) < 0$ (menghasilkan refleksi, bukan rotasi murni), koreksi dengan membalik tanda baris pertama $V^\top$:

$$\text{if } \det(R) < 0 : \quad V_{0,:}^\top \leftarrow -V_{0,:}^\top, \quad R = UV^\top$$

Langkah 6: Translasi Optimal

$$t = \bar{m} - R\bar{s}$$

Langkah 7: Susun Rigid Transform 2D

$$T_{\text{update}} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \Delta\theta & -\sin \Delta\theta & t_x \\ \sin \Delta\theta & \cos \Delta\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Setelah iterasi, cek apakah transform update cukup kecil:

$$\text{converged} \iff \|t\| < t_{\text{norm}} \wedge |\arctan 2(R_{1,0}, R_{0,0})| < r_{\text{norm}}$$

Default: $t_{\text{norm}} = 0.01$ m, $r_{\text{norm}} = 0.01$ rad ($\approx 0.57^\circ$).

Untuk mempercepat, hanya $1/s$ dari scan points yang digunakan:

1. Pilih offset acak $o \in [0, s - 1]$ 2. Ambil setiap point ke- s mulai dari o :

$$p'_k = p_{o+k \cdot s}, \quad k = 0, 1, \dots, \left\lfloor \frac{N}{s} \right\rfloor - 1$$

Default $s = 3$

3.2.2 Costmap

Costmap adalah representasi grid yang menyimpan informasi biaya untuk navigasi. Setiap sel dalam costmap memiliki nilai biaya yang mencerminkan kesulitan atau risiko melewati area tersebut. Nilai biaya ini dapat dipengaruhi oleh berbagai faktor seperti keberadaan rintangan. Costmap menyimpan informasi map grid 2D dalam bentuk array 1D

Tabel 3.1: Nilai konstanta costmap

Nama Konstanta	Nilai	Makna
NO_INFORMATION	255	Tidak diketahui (<i>unknown</i>)
LETHAL_OBSTACLE	254	Obstacle mematikan
INSCRIBED_INFLATED_OBSTACLE	253	Batas terluar inflasi obstacle
INFLATED_OBSTACLE	6-252	Inflasi obstacle
TRANSITION_CELL	1-5	Area transisi antar region
FREE_SPACE	0	Ruang bebas

Indeks Grid

konversi dari koordinat grid 2D (m_x, m_y) ke flat index 1D:

$$i = m_y \cdot W + m_x$$

dimana W adalah lebar grid (jumlah cell per baris).

Koordinat Dunia - Grid

Diberikan origin grid di (o_x, o_y) dan resolusi r (meter per cell):

****World → Grid:****

$$m_x = \left\lfloor \frac{w_x - o_x}{r} \right\rfloor, \quad m_y = \left\lfloor \frac{w_y - o_y}{r} \right\rfloor$$

****Grid → World** :**

$$w_x = o_x + \left(m_x + \frac{1}{2}\right) \cdot r, \quad w_y = o_y + \left(m_y + \frac{1}{2}\right) \cdot r$$

1.4 Rolling Window

Untuk local costmap yang bergerak mengikuti robot, origin grid digeser:

$$\Delta_x = \lfloor \text{new_origin}_x - \text{old_origin}_x \rfloor, \quad \Delta_y = \lfloor \text{new_origin}_y - \text{old_origin}_y \rfloor$$

Array costmap digeser sebesar

$$(\Delta_x, \Delta_y)$$

dan cell baru diisi dengan NO_INFORMATION.

3.2.2.1 Static Layer

Static layer adalah lapisan costmap paling dasar yang menyimpan informasi *known obstacle* atau tembok yang sudah diketahui. Lapisan costmap ini mengkonversi setiap sel dari nilai occupancy grid (0-100) ke nilai biaya costmap internal (0-255). Sel yang tidak diketahui (unknown) diberi biaya tertinggi, sel bebas diberi biaya terendah, dan sel yang terisi (occupied) diberi biaya tinggi.

$$C_{\text{internal}} = \begin{cases} 255 & \text{if } OG = -1 \text{ (unknown)} \\ 0 & \text{if } OG = 0 \text{ (free)} \\ 254 & \text{if } OG \geq 100 \text{ (occupied)} \end{cases}$$

3.2.2.2 Obstacle Layer

Obstacle layer dirancang untuk menangani objek-objek dinamis atau rintangan yang baru terdeteksi di lingkungan. Informasi pada lapisan ini diperoleh secara real-time dari sensor LiDAR robot, yang terus-menerus memindai area sekitar dan memperbarui *costmap* sesuai dengan obstacle yang terdeteksi. Lapisan ini memiliki batasan jangkauan observasi maksimum, yang jaraknya disesuaikan berdasarkan jarak *pointcloud* maksimum dari sensor LiDAR, yang berarti hanya rintangan dalam radius pandang sensor yang akan direpresentasikan di lapisan *costmap*

ini.

Untuk setiap beam laser dengan range r dan sudut θ :

1. Hitung endpoint dalam frame sensor: $(r \cos \theta, r \sin \theta)$
2. Transform ke frame map via TF
3. Konversi ke cell: $(m_x, m_y) = \text{worldToMap}(x, y)$
4. Set cell ke LETHAL_OBSTACLE (254)

Gunakan algoritma ****Bresenham 2D**** untuk menggambar garis dari robot ke endpoint sensor. Semua cell yang dilewati garis (sebelum endpoint) di-clear:

$$C_{\text{master}}(m_x, m_y) = 0 \quad \forall \text{cell di sepanjang ray (kecuali endpoint)}$$

Bresenham 2D Raytracing

Algoritma Bresenham memilih pixel-pixel yang mendekati garis lurus antara dua titik grid. Untuk setiap langkah, algoritma memutuskan apakah akan bergerak di sumbu x, y, atau keduanya berdasarkan error accumulator:

Diberikan titik awal (x_0, y_0) dan titik akhir (x_1, y_1) :

$$\Delta_x = |x_1 - x_0|, \quad \Delta_y = |y_1 - y_0|$$

$$\text{err} = \Delta_x - \Delta_y, \quad s_x = \text{sign}(x_1 - x_0), \quad s_y = \text{sign}(y_1 - y_0)$$

Algorithm 2 Bresenham Raytracing

```

1: while  $x \neq x_1 \vee y \neq y_1$  do
2:    $e_2 \leftarrow 2 \cdot \text{err}$ 
3:   if  $e_2 > -\Delta_y$  then
4:      $\text{err} \leftarrow \text{err} - \Delta_y$ 
5:      $x \leftarrow x + s_x$ 
6:   end if
7:   if  $e_2 < \Delta_x$  then
8:      $\text{err} \leftarrow \text{err} + \Delta_x$ 
9:      $y \leftarrow y + s_y$ 
10:  end if
11:  raytrace_function( $x, y$ )
12: end while

```

3.2.2.3 Inflation Layer

Inflation layer berperan penting dalam memastikan pergerakan robot dapat menghindari berhimpitan dengan rintangan. Lapisan ini menciptakan zona inflasi di sekitar setiap rintangan

yang terdeteksi dan memiliki nilai harga navigasi yang tinggi. Radius zona inflasi ini variabel yang ditentukan berdasarkan dimensi fisik atau *footprint* robot sehingga mencegah robot terlalu dekat dengan rintangan dan berpotensi menabrak. Lapisan ini secara efektif ”memperbesar” rintangan di *costmap*, sehingga algoritma perencanaan jalur akan menjaga jarak aman dari objek-objek tersebut dari robot.

Lapisan inflasi berfungsi seperti safe distance minimum robot dari rintangan, namun dapat bekerja lebih dinamis karena memperhitungkan orientasi dan bentuk badan robot. Dengan bentuk robot yang unik, radius zona inflasi dapat berubah secara adaptif sesuai orientasi robot terhadap rintangan.

Layer ini memperluas obstacle dengan gradient cost, sehingga perencana path dapat menjaga jarak dari obstacle.

Algoritma Inflasi

Untuk setiap obstacle cell, terapkan cost ke semua cell dalam radius inflasi R .

Cost pada jarak d dari obstacle dihitung dengan fungsi eksponensial:

$$C(d) = \begin{cases} 253 \cdot e^{-\alpha \cdot (d - r_{\text{inscribed}})} & \text{if } d \leq R \\ 0 & \text{otherwise} \end{cases}$$

Atau dalam bentuk weight:

$$C(d) = 252 \cdot \left(e^{-\alpha \cdot (d - r_{\text{inscribed}})} \right)$$

dimana: - d = jarak Euclidean dari cell ke obstacle terdekat - R = inflation radius (jarak maksimum inflasi) - $r_{\text{inscribed}}$ = inscribed radius (radius robot, cost = 253 pada jarak ini) - α = scaling factor (mengontrol kecuraman kurva cost)

Cost dibulatkan ke integer 0–252.

3.2.2.4 Transition Layer

Transition layer adalah lapisan yang menambahkan data area transisi ke dalam *costmap*. Data ini digunakan oleh untuk mendeteksi kapan robot mendekati batas transisi antar region dan melakukan ”lompatan” pose antar area.

$$C_{\text{internal}} = \begin{cases} 255 & \text{if } OG = -1 \text{ (unknown)} \\ 0 & \text{if } OG = 0 \text{ (free)} \\ 1 & \text{if } OG \geq 100 \text{ (occupied)} \end{cases}$$

Lapisan ini juga memperluas area transisi dengan radius tertentu, sehingga robot dapat mendeteksi transisi sebelum benar-benar mencapai garis transisi. Hal ini membuat buffer zona deteksi di sekitar data transisi.

Tiap region di map memiliki metadata transisi unik yang menyimpan informasi relasi antar peta. Informasi tersebut termasuk orientasi relatif terhadap orientasi global yang dijadikan acuan

3.2.3 Local Planner

DWA adalah algoritma **local path planning** yang bekerja dengan cara:

DWA — Velocity window dibatasi oleh apa yang **dapat dicapai dalam satu control cycle**:

$$v_{\min} = \max(v_{\min.\text{limit}}, v_{\text{current}} - \dot{v}_{\max} \cdot \Delta t)$$

$$v_{\max} = \min(v_{\max.\text{limit}}, v_{\text{current}} + \dot{v}_{\max} \cdot \Delta t)$$

dimana Δt adalah sim period = 0.05 s (waktu satu siklus kontrol).

Trajectory Rollout — Velocity window dibatasi oleh apa yang dapat dicapai **selama waktu simulasi penuh**:

$$v_{\max} = \min(v_{\max.\text{limit}}, v_{\text{current}} + \dot{v}_{\max} \cdot T_{\text{sim}})$$

dimana T_{sim} adalah sim time

2.2 Velocity Sampling

Untuk setiap dimensi (v_x, v_y, ω) , rentang dibagi menjadi sample merata:

$$\Delta v_x = \frac{v_{x,\max} - v_{x,\min}}{N_{v_x} - 1}$$

$$\Delta v_y = \frac{v_{y,\max} - v_{y,\min}}{N_{v_y} - 1}$$

$$\Delta \omega = \frac{\omega_{\max} - \omega_{\min}}{N_{\omega} - 1}$$

Total trajectory yang di-sample:

$$N_{\text{total}} = N_{v_x} \times N_{v_y} \times N_{\omega}$$

Konfigurasi dari yaml: $N_{v_x} = 6, N_{v_y} = 6, N_{\omega} = 30 \rightarrow$ total 1080 trajectories per evaluasi.

3.1 Model Kinematik

Untuk setiap sample kecepatan (v_x, v_y, ω) , trajectory disimulasikan menggunakan model kinematik diferensial:

$$x_{t+1} = x_t + \left(v_x \cos \theta_t + v_y \cos \left(\frac{\pi}{2} + \theta_t \right) \right) \cdot \Delta t$$

$$y_{t+1} = y_t + \left(v_x \sin \theta_t + v_y \sin \left(\frac{\pi}{2} + \theta_t \right) \right) \cdot \Delta t$$

$$\theta_{t+1} = \theta_t + \omega \cdot \Delta t$$

Jumlah langkah simulasi ditentukan oleh granularity:

$$N_{\text{steps}} = \frac{T_{\text{sim}}}{\Delta t}$$

$$\Delta t = \min\left(\frac{\text{sim_granularity}}{|v|}, \frac{\text{angular_sim_granularity}}{|\omega|}\right)$$

Trajectory Scoring

$$C_{\text{total}} = C_{\text{osc}} + w_{\text{obs}} \cdot C_{\text{obs}} + w_{\text{gf}} \cdot C_{\text{gf}} + w_{\text{align}} \cdot C_{\text{align}} + w_{\text{path}} \cdot C_{\text{path}} + w_{\text{goal}} \cdot C_{\text{goal}} + w_{\text{twirl}} \cdot C_{\text{twirl}}$$

$$C_{\text{obs}} = \begin{cases} \sum_{k=1}^N C_k & \text{if sum_scores} \\ \max_k C_k & \text{otherwise} \end{cases}$$

5.2.2 Collision Checking

Setiap titik sepanjang trajectory, footprint robot di-rasterize ke costmap:

Menggunakan Bresenham line drawing untuk menelusuri setiap tepi footprint polygon di grid costmap.

5.2.3 Agregasi Cost

Mode sum scores:

$$C_{\text{obs}} = \begin{cases} \sum_{k=1}^N C_k & \text{if sum_scores} \\ \max_k C_k & \text{otherwise} \end{cases}$$

3.2.3.1 MapGridCostFunction

****Tujuan**:** Menilai trajectory berdasarkan jarak ke global plan atau goal

.3.1 Wavefront Propagation

‘MapGrid’ menggunakan BFS untuk menyebarkan nilai jarak dari target cells:

“ 1. Reset semua cell ke obstacleCosts() 2. Set target cells (path waypoints atau goal point) → distance = 0 3. BFS: setiap tetangga yang bukan obstacle mendapat distance = parent distance + 1 4. Berhenti di LETHAL OBSTACLE, INSCRIBED INFLATED OBSTACLE, NO INFORMATION “

$$d_{\text{cell}} = \text{number of BFS steps from nearest target cell}$$

5.3.2 Scoring Mode

Untuk setiap titik pada trajectory, nilai ‘target dist’ dari MapGrid dibaca.

****Aggregation type****

— Mode — Formula — Kegunaan —

— ‘Last’ — $C = d_N$ (jarak di titik terakhir) — Goal cost —

— ‘Sum’ — $C = \sum_{k=1}^N d_k$ — Path cost —

Forward Point Shift

$$p_{\text{score}} = p_{\text{robot}} + d_{\text{forward}} \cdot \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}$$

Path Costs (C_{path})

Weight: $w_{\text{path}} = \text{resolution} \cdot \text{path_distance_bias}$

Goal Costs (C_{goal})

Weight: $w_{\text{goal}} = \text{resolution} \cdot \text{goal_distance_bias}$

Goal Front Costs (C_{gf})

Menggunakan forward point shift. Menilai apakah ****hidung robot**** mengarah ke goal.

$$p_{\text{hidung}} = p_{\text{robot}} + d_{\text{forward}} \cdot \hat{v}$$

$$C_{\text{gf}} = \text{target_dist}(p_{\text{hidung}})$$

Alignment Costs (C_{align})

Sama dengan path costs, tetapi dengan forward point shift. Menilai apakah hidung robot sejajar dengan path.

Dinonaktifkan jika robot sudah dekat dengan goal:

$$\text{disabled if } \|p_{\text{robot}} - p_{\text{goal}}\| < d_{\text{forward}} \cdot \text{cheat_factor}$$

Best Trajectory Selection

```
1 1. Prepare semua critics:
2   - MapGridCostFunction::prepare() -> wavefront BFS propagation
3 2. Iterate semua trajectory dari generator:
4   a. GenerateTrajectory(pos, vel, sample_vel, &traj)
5   b. scoreTrajectory(traj, best_cost)
6   c. Jika cost >= 0 dan cost < best_cost -> update best trajectory
7 3. Jika tidak ada trajectory valid -> gunakan zero velocity
```

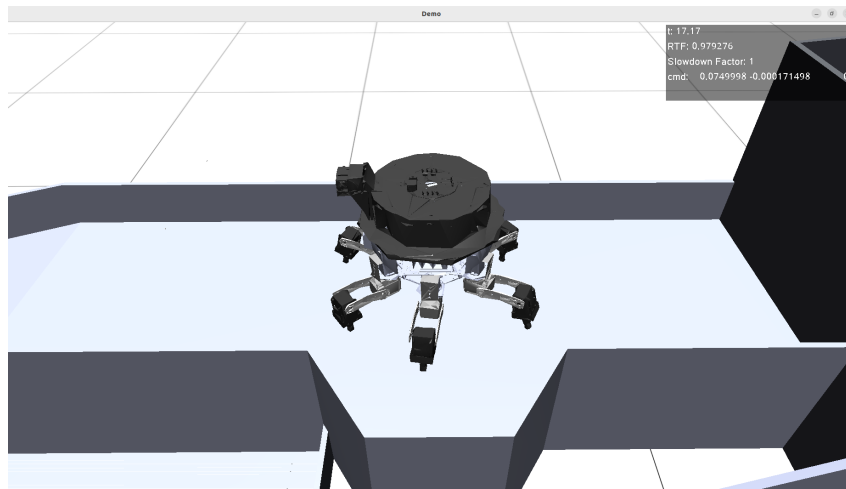
Early termination: Jika total cost sudah melebihi best_cost saat menjumlahkan critics, evaluasi trajectory dihentikan lebih awal.

BAB IV

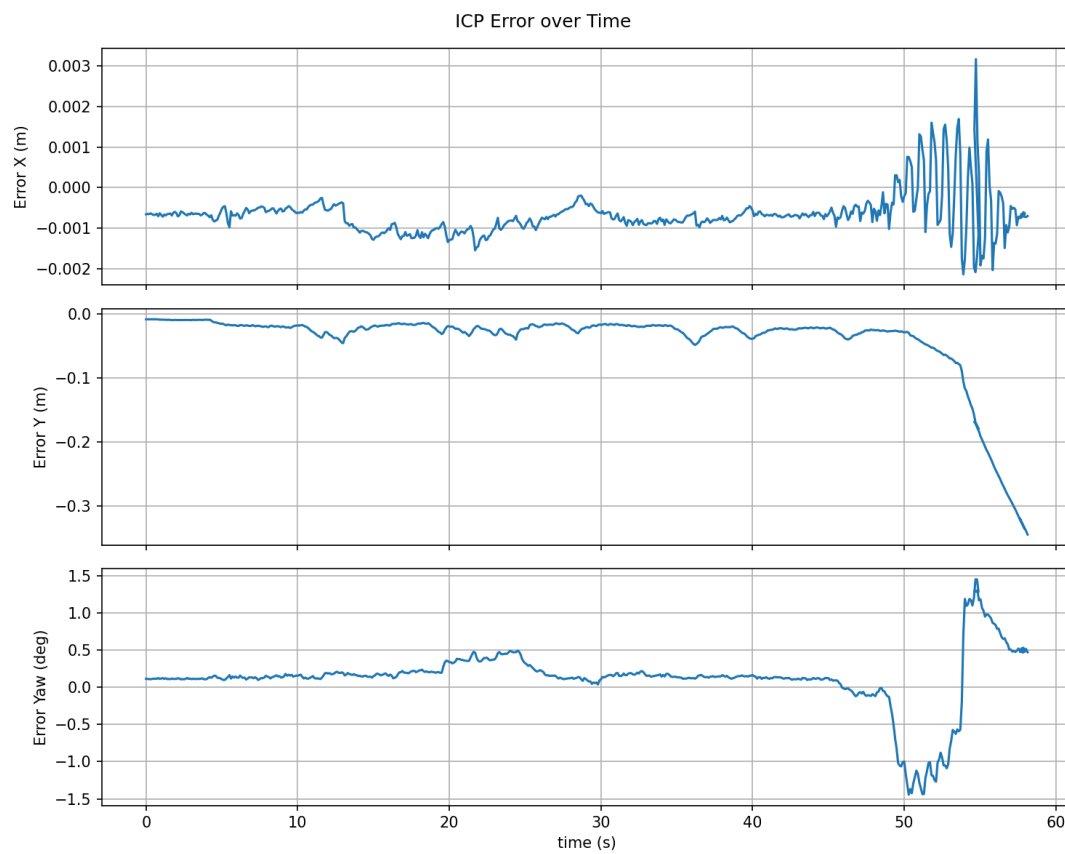
HASIL DAN PEMBAHASAN

4.1 Pengujian dan Hasil

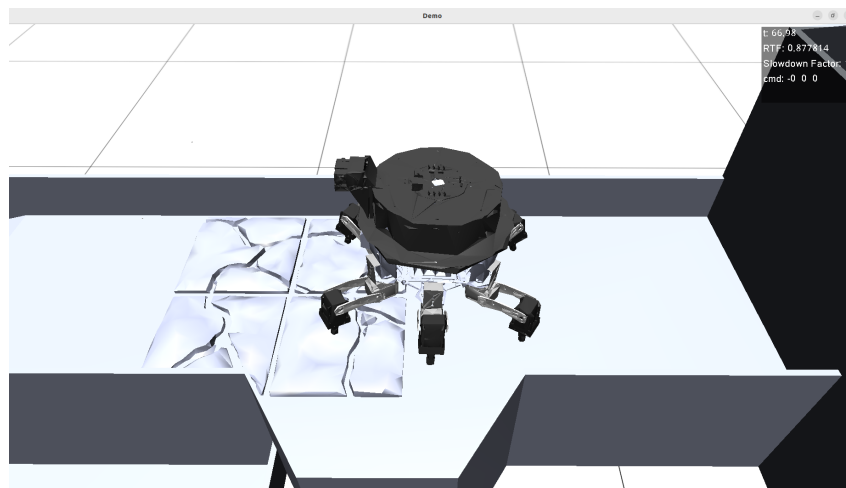
4.1.1 Hasil Lokalisasi ICP



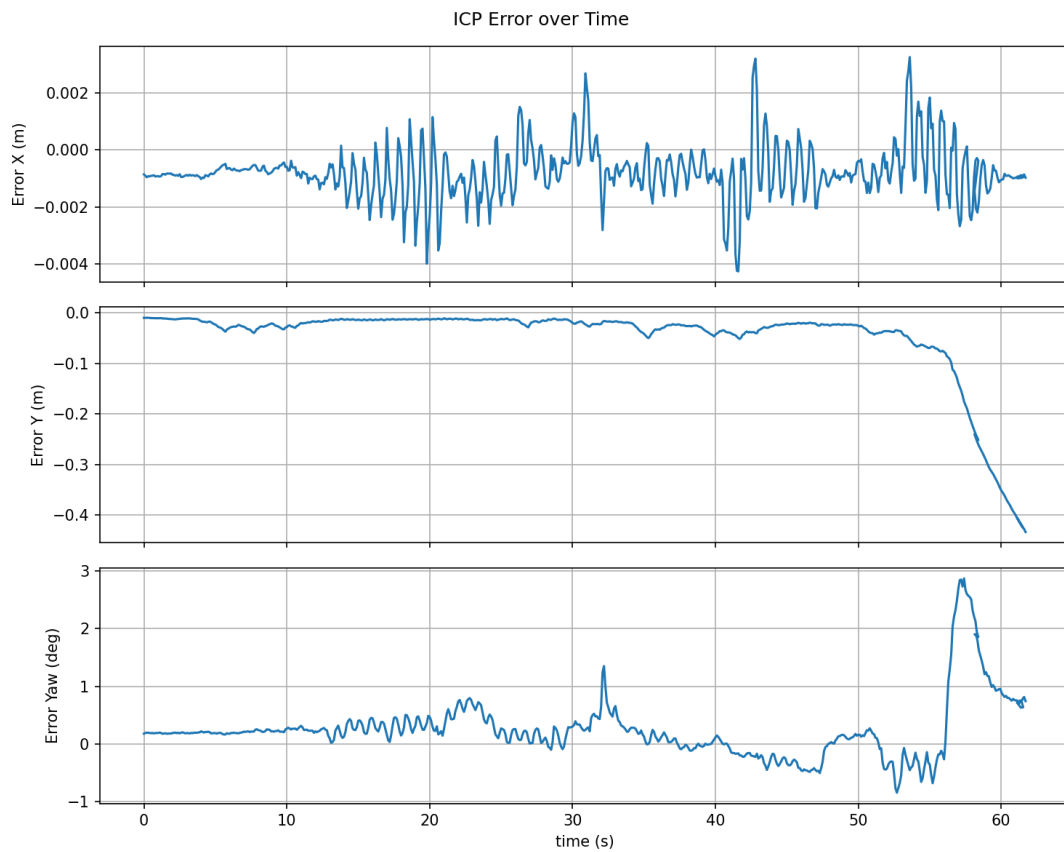
Gambar 4.1: Simulasi di Region A



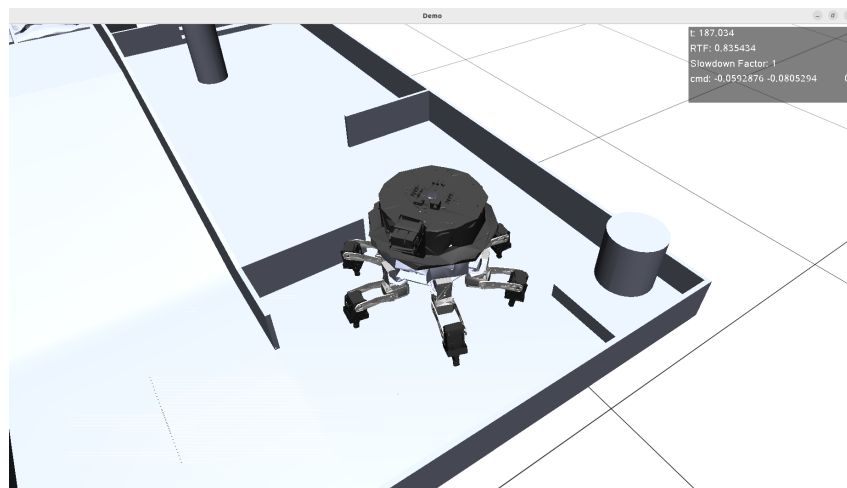
Gambar 4.2: Error ICP di Region A



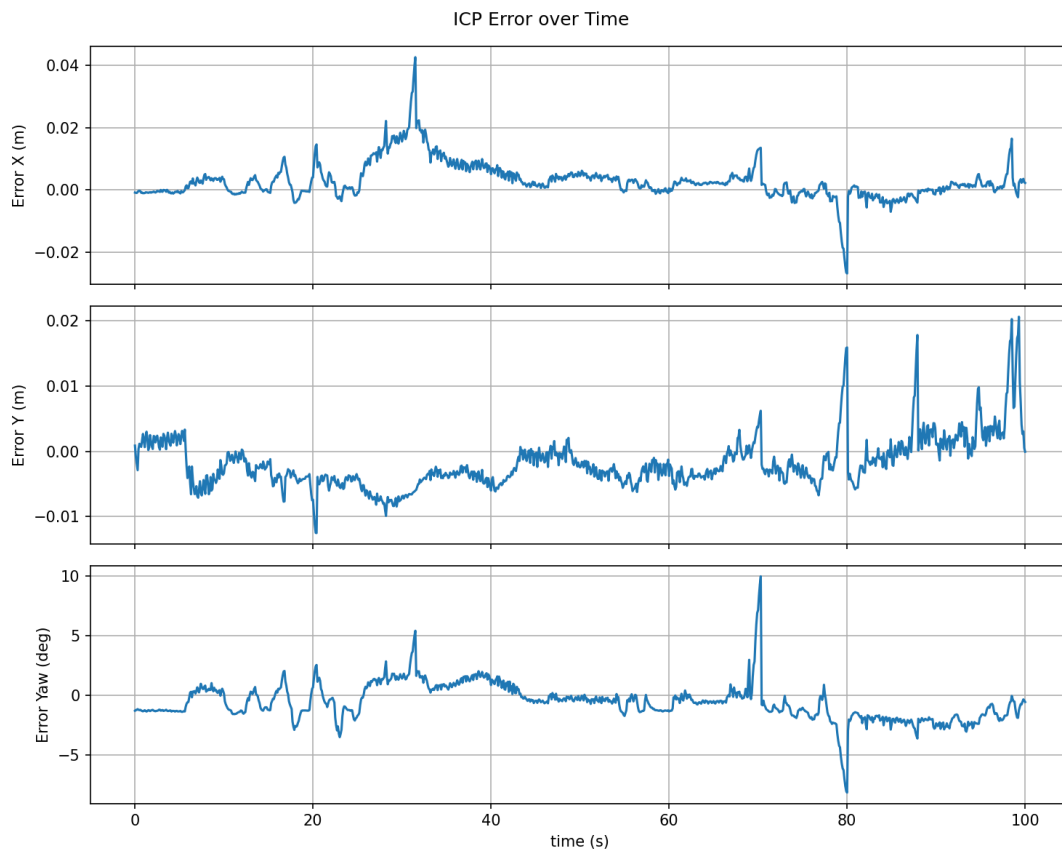
Gambar 4.3: Simulasi di Region A dengan rintangan rantai



Gambar 4.4: Error ICP di Region A dengan rintangan lantai



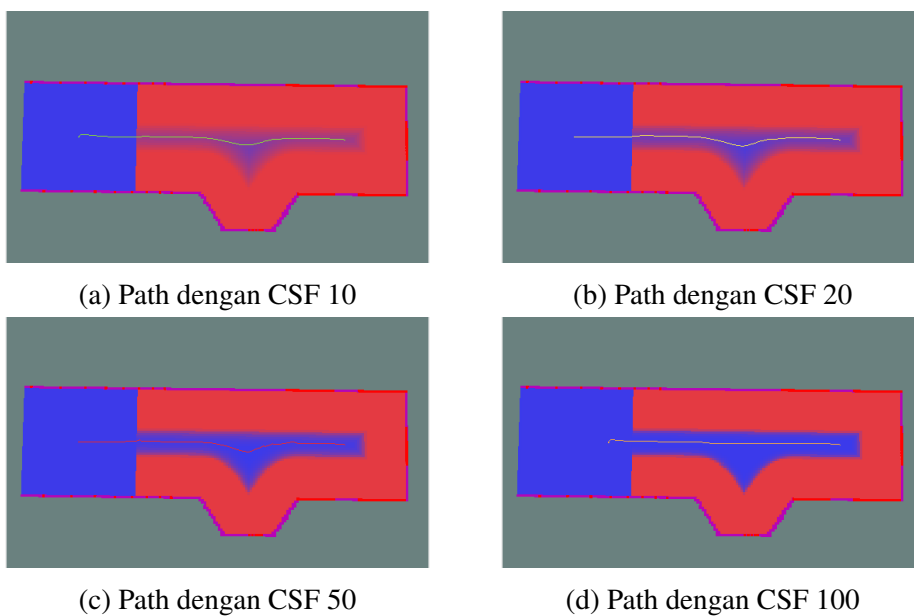
Gambar 4.5: Simulasi di Region C



Gambar 4.6: Error ICP di Region C

4.1.2 Hasil Global Planner

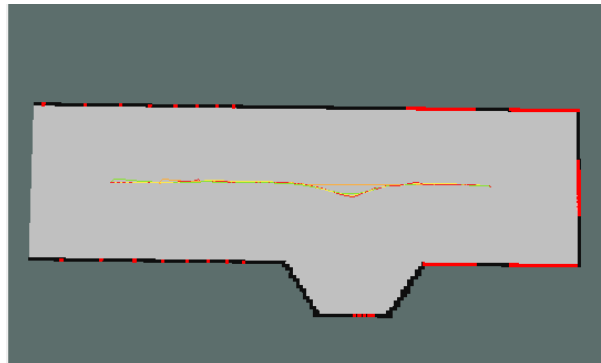
4.1.2.1 Hasil Path di Region A



Gambar 4.7: Relasi path terhadap Cost Scaling Factor di Region A

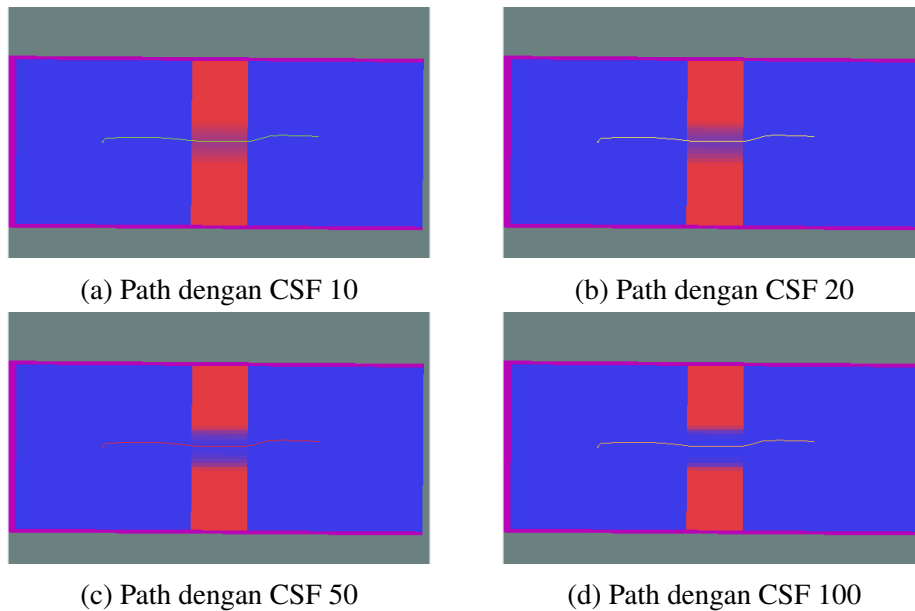
Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	1.116051	6.666027	0.220000	222
20	1.114836	6.897399	0.220000	223
50	1.122597	5.688345	0.219650	225
100	0.971836	6.484874	0.216470	193

Tabel 4.1: Karakteristik Path di Region A berdasarkan Cost Scaling Factor



Gambar 4.8: Semua path di region A.
 ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100

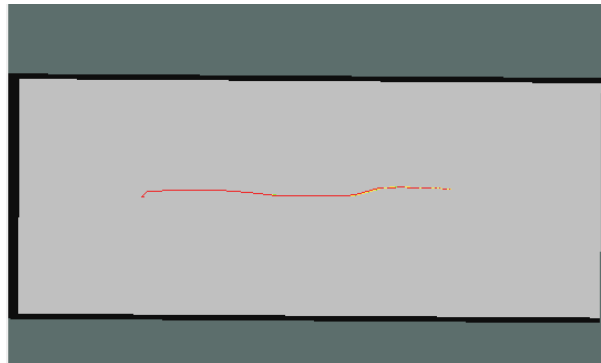
4.1.2.2 Hasil Path di Region B



Gambar 4.9: Relasi path terhadap Cost Scaling Factor di Region B

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	0.591730	152.256312	0.208163	118
20	0.591773	135.818868	0.208066	118
50	0.591600	20.557631	0.208224	118
100	0.591590	10.574465	0.208132	118

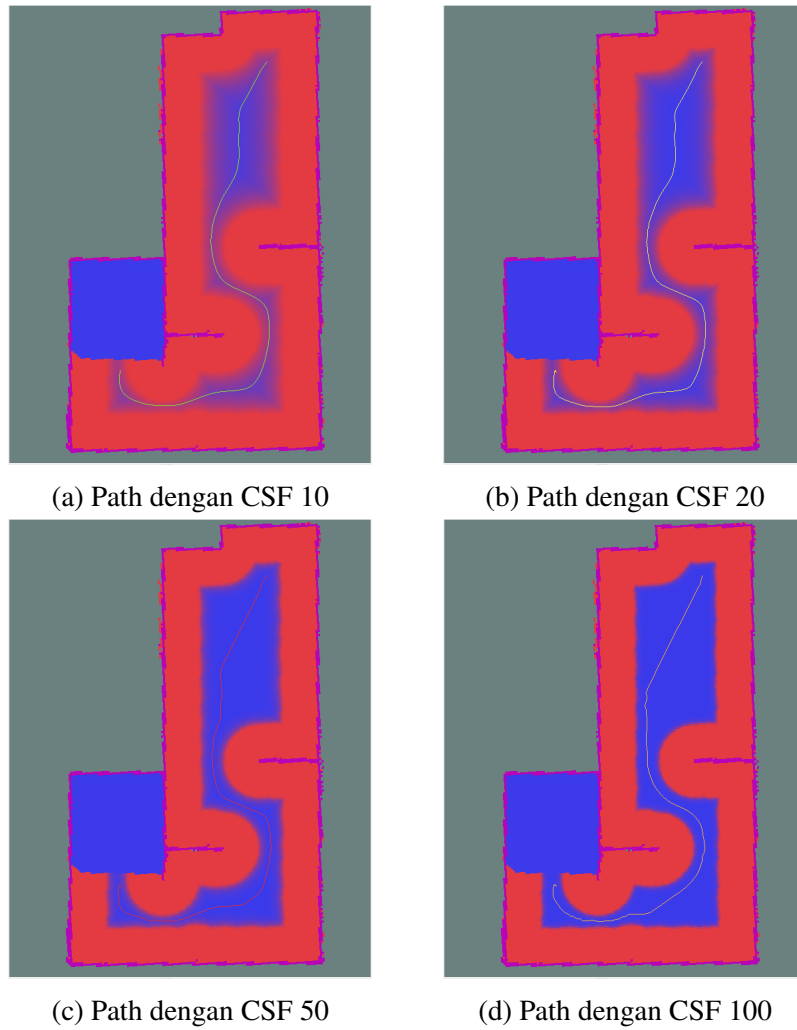
Tabel 4.2: Karakteristik Path di Region B berdasarkan Cost Scaling Factor



Gambar 4.10: Semua path di region B.

● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100

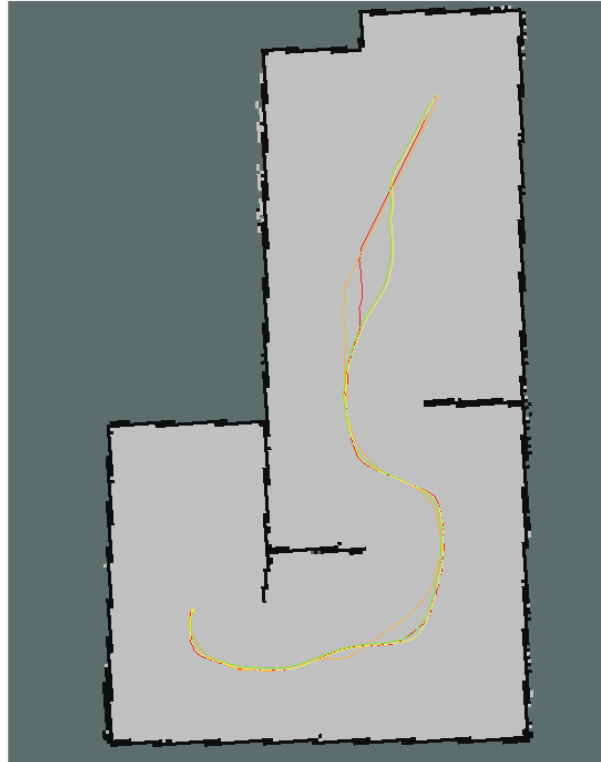
4.1.2.3 Hasil Path di Region C



Gambar 4.11: Relasi path terhadap Cost Scaling Factor di Region C

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	2.542122	10.629525	0.197048	509
20	2.596590	10.268101	0.196292	521
50	2.596873	9.922248	0.197185	520
100	2.524643	10.224478	0.195499	506

Tabel 4.3: Karakteristik Path di Region C berdasarkan Cost Scaling Factor



Gambar 4.12: Semua path di region C.
 ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100

Cost Scaling Factor	Rata-rata panjang Path (m)	Jumlah waypoint	Mean cross track error (m)	Standard Deviation (m)	Max Cross track Error (m)
20	1.11483	223	0.002219	0.001795	0.009989
50	1.12259	225	0.002061	0.002086	0.009949
100	0.97183	193	0.005290	0.007304	0.026245

Tabel 4.4: Perbandingan path terhadap ground truth di region A

Cost Scaling Factor	Rata-rata panjang Path (m)	Jumlah waypoint	Mean cross track error (m)	Standard Deviation (m)	Max Cross track Error (m)
20	0.59177	118	0.000050	0.000075	0.000234
50	0.59159	118	0.000231	0.000527	0.002048
100	0.59164	118	0.000234	0.000545	0.002056

Tabel 4.5: Perbandingan path terhadap ground truth di region B

Cost Scaling Factor	Rata-rata panjang Path (m)	Jumlah waypoint	Mean cross track error (m)	Standard Deviation (m)	Max Cross track Error (m)
20	2.59641	521	0.005548	0.002228	0.013089
50	2.60291	522	0.012802	0.018897	0.089070
100	2.53762	509	0.020026	0.024742	0.099313

Tabel 4.6: Perbandingan path terhadap ground truth di region C

4.2 Pembahasan

[Halaman ini sengaja dikosongkan]

BAB V

PENUTUP

5.1 Kesimpulan

5.2 Saran

[Halaman ini sengaja dikosongkan]

DAFTAR PUSTAKA

- Damle, V. P., & Susan, S. (2022). Dynamic algorithm for path planning using a-star with distance constraint. *2022 2nd International Conference on Intelligent Technologies (CONIT)*, 1–5.
- Hu, B., Cui, M., & Cao, Z. (2022). A slope-adaptive navigation approach for ground mobile robots. *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 610–615.
- Jung, D., Kim, C., Cho, J.-K., & Kim, S.-W. (2024). Munes: Multifloor navigation including elevators and stairs. *arXiv preprint arXiv:2402.04535*.
- Kim, E.-H., Bae, S.-H., & Kuc, T.-Y. (2020). Mobile service robot multi-floor navigation using visual detection and recognition of elevator features (iccas 2020). *2020 20th International Conference on Control, Automation and Systems (ICCAS)*, 982–985.
- Kim, T., Kang, G., Lee, D., & Shim, D. H. (2024). Development of an indoor delivery mobile robot for a multi-floor environment. *IEEE Access*.
- Lee, J. Y., Tokuda, T., & Sato, K. (2023). Autonomous floor navigation control of mobile robot using elevator button recognition with deep learning. *2023 62nd Annual Conference of the Society of Instrument and Control Engineers (SICE)*, 133–138.
- Li, S., Chen, Y., Meng, Y., & Lou, Y. (2021). Autonomous elevator button recognition and operation framework for multi-floor mobile manipulator navigation. *2021 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 383–388.
- Lin, T.-H., Huang, J.-T., & Putranto, A. (2022). Integrated smart robot with earthquake early warning system for automated inspection and emergency response. *Natural hazards*, *110*(1), 765–786.
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., & Ferguson, M. (2023). From the desks of ros maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, *168*, 104493.
- Palacín, J., Bitriá, R., Rubies, E., & Clotet, E. (2023). A procedure for taking a remotely controlled elevator with an autonomous mobile robot based on 2d lidar. *Sensors*, *23*(13), 6089.
- Palacín, J., Rubies, E., Bitriá, R., & Clotet, E. (2023). Path planning of a mobile delivery robot operating in a multi-story building based on a predefined navigation tree. *Sensors*, *23*(21), 8795.
- Sabtaji, A. (2020). Statistik kejadian gempa bumi tektonik tiap provinsi di wilayah indonesia selama 11 tahun pengamatan (2009-2019). *Buletin Meteorologi, Klimatologi, dan Geofisika*, *1*(7), 31–46.
- Sobreira, H., Costa, C. M., Sousa, I., Rocha, L., Lima, J., Farias, P., Costa, P., & Moreira, A. P. (2019). Map-matching algorithms for robot self-localization: A comparison between perfect match, iterative closest point and normal distributions transform. *Journal of Intelligent & Robotic Systems*, *93*, 533–546.
- Xie, H., & Zhou, Q. (2023). Autonomous traversing across floors based on vision for reconfigurable tracked robot. *2023 8th International Conference on Control, Robotics and Cybernetics (CRC)*, 14–19.

- Yao, Q., Tian, Y., Wang, Q., & Wang, S. (2020). Control strategies on path tracking for autonomous vehicle: State of the art and future challenges. *IEEE Access*, 8, 161211–161222.
- Zhang, Y., Li, Y., Zhang, H., Wang, Y., Wang, Z., Ye, Y., Yue, Y., Guo, N., Gao, W., Chen, H., et al. (2023). Earthshaker: A mobile rescue robot for emergencies and disasters through teleoperation and autonomous navigation. *JUSTC*, 53(1), 4–1.
- Zhu, K., Zhan, J., Chen, S., Nan, Z., Zhang, T., Zhu, D., & Zheng, N. (2020). Atv navigation in complex and unstructured environment containing stairs. *2020 IEEE Intelligent Vehicles Symposium (IV)*, 817–824.

LAMPIRAN

ddd

[Halaman ini sengaja dikosongkan]

BIOGRAFI PENULIS

[Halaman ini sengaja dikosongkan]